Check for updates

ACM DIGITAL LIBRARY

Association for Computing Machinery

acm open

Latest updates: https://dl.acm.org/doi/10.1145/3422392.3422437

PDF Download
3422392.3422437.pdf
01 March 2026
Total Citations: 9
Total Downloads: 215

Published: 21 December 2020

Citation in BibTeX format

SBES '20: 34th Brazilian Symposium on Software Engineering
October 21 - 23, 2020
Natal, Brazil

RESEARCH-ARTICLE

Atoms of Confusion: The Eyes Do Not Lie

BENEDITO DE OLIVEIRA, Federal University of Alagoas, Maceio, AL, Brazil

MÁRCIO RIBEIRO, Federal University of Alagoas, Maceio, AL, Brazil

JOSÉ ALDO SILVA DA COSTA, Federal University of Campina Grande, Campina Grande, PB, Brazil

ROHIT GHEYI, Federal University of Campina Grande, Campina Grande, PB, Brazil

GUILHERME AMARAL, Federal University of Alagoas, Maceio, AL, Brazil

RAFAEL DE MELLO, Federal Center for Technological Education Celso Suckow da Fonseca, Rio de Janeiro, RJ, Brazil

View all

Open Access Support provided by:

Federal University of Alagoas

University of Brasilia

Pontifical Catholic University of Rio de Janeiro

Federal University of Campina Grande

Federal Center for Technological Education Celso Suckow da Fonseca

SBES '20: Proceedings of the XXXIV Brazilian Symposium on Software Engineering (October 2020)
https://doi.org/10.1145/3422392.3422437
ISBN: 9781450387538

Atoms of Confusion: The Eyes Do Not Lie

Benedito de Oliveira
Federal University of Alagoas
Maceió, Alagoas, Brazil
bfao@ic.ufal.br

Márcio Ribeiro
Federal University of Alagoas
Maceió, Alagoas, Brazil
marcio@ic.ufal.br

José Aldo Silva da Costa
Federal University of Campina Grande
Campina Grande, Paraíba, Brazil
josealdo@copin.ufcg.edu.br

Rohit Gheyi
Federal University of Campina Grande
Campina Grande, Paraíba, Brazil
rohit@dsc.ufcg.edu.br

Guilherme Amaral
Federal University of Alagoas
Maceió, Alagoas, Brazil
gvma@ic.ufal.br

Rafael de Mello
Federal Center for Technological Education of Rio de Janeiro
Rio de Janeiro, Rio de Janeiro, Brazil
rafael.mello@cefet-rj.br

Anderson Oliveira
PUC-Rio
Rio de Janeiro, Rio de Janeiro, Brazil
aoliveira@inf.puc-rio.br

Alessandro Garcia
PUC-Rio
Rio de Janeiro, Rio de Janeiro, Brazil
afgarcia@inf.puc-rio.br

Rodrigo Bonifácio
University of Brasília
Brasília, Brazil
rbonifacio@unb.br

Baldoino Fonseca
Federal University of Alagoas
Maceió, Alagoas, Brazil
baldoino@ic.ufal.br

ABSTRACT

Code comprehension is crucial in software maintenance activities, though it can be hindered by misunderstandings and confusion patterns, namely, atoms of confusion. They are small pieces of code using specific programming language constructs, such as *Conditional Operators* and *Comma Operators*. A previous study showed that these atoms of confusion impact developers' performance, i.e., time and accuracy, and increase code misunderstandings. However, empirical knowledge of the impact of such atoms on code comprehension is still scarce, especially when it comes to analyzing that impact on developers' visual attention. The present study evaluates whether developers misunderstand the code in the presence of atoms of confusion with an eye tracker. For this purpose, we measure the tasks' completion time, the accuracy of the answers, and analyze the distribution of visual attention. We conducted a controlled experiment with 30 students and software practitioners. We ask the subjects to specify the output of three tasks with atoms and three without atoms randomly assigned using a Latin Square design. We use an eye-tracking camera to detect the visual attention of the participants while solving the tasks. From an aggregated perspective, we observed an increase of 43.02% in time and 36.8% in gaze transitions in code snippets with atoms. For accuracy, no statistically significant difference was observed. We also confirm that the regions that receive most of the eye attention were the regions with atoms. Our findings reinforce that atoms hinder developers' performance and comprehension. So, developers should avoid writing code with them.

ACM acknowledges that this contribution was authored or co-authored by an employee, contractor or affiliate of a national government. As such, the Government retains a nonexclusive, royalty-free right to publish or reproduce this article, or to allow others to do so, for Government purposes only.
SBES '20, October 21–23, 2020, Natal, Brazil
© 2020 Association for Computing Machinery.
ACM ISBN 978-1-4503-8753-8/20/09…$15.00
https://doi.org/10.1145/3422392.3422437

CCS CONCEPTS

• **Software and its engineering → Maintaining software**; *Software verification and validation*; Abstraction, modeling and modularity.

KEYWORDS

Atoms of Confusion, Code Comprehension, Eye Tracking

ACM Reference Format:
Benedito de Oliveira, Márcio Ribeiro, José Aldo Silva da Costa, Rohit Gheyi, Guilherme Amaral, Rafael de Mello, Anderson Oliveira, Alessandro Garcia, Rodrigo Bonifácio, and Baldoino Fonseca. 2020. Atoms of Confusion: The Eyes Do Not Lie. In *34th Brazilian Symposium on Software Engineering (SBES '20), October 21–23, 2020, Natal, Brazil.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3422392.3422437

1 INTRODUCTION

Code comprehension is a critical activity in software development, especially in the maintenance and evolution processes. Developers often have to deal with maintaining or improving code that they did not write. To perform any modification in the code, first, developers have to understand it. Indeed, a previous study showed that most of their time is spent on code comprehension activities [13]. However, the comprehension process can be hindered by aspects of the code that cause misunderstandings. For instance, previous works [4, 5, 22] have successfully identified a set of tiny code patterns that contribute to the increase of the time and effort necessary

to understand the code correctly. Examples of these small code patterns in imperative languages like C and C++, namely *atoms of confusion*, include *Conditional Operators*, *Comma Operators*, *Logic as Control Flow*, among others, and are often found in source code bases [10].

In a previous study [4], researchers conducted a controlled experiment to compare the performance, i.e., time and accuracy of participants when dealing with code with and without atoms of confusion. They have shown that the presence of atoms of confusion in the code increases misunderstandings and that. However, empirical knowledge on the impact of such atoms on code comprehension is still scarce, given the difficulties in measuring code comprehension [18]. On the other hand, the code comprehension field has been gaining new insights from other dimensions besides time and accuracy with eye-tracking devices [18]. For instance, knowing which parts of the code receive more or less attention would give us insights on the extent of the impact of such atoms on visual effort, which is a dimension not considered before [4] or in any other study that we are aware. Although time and accuracy are good estimators of code confusion, knowing precisely the regions that participants paid attention, allows us to estimate their real effects.

Given this scenario, we use an eye tracker to evaluate whether developers misunderstand the code in the presence of atoms of confusion. For this purpose, we measure time, accuracy, and distinctly from other studies, we also analyze the visual effort given to specific regions in the code. The triangulation of these dimensions allows us to understand better the real effects of atoms of confusion in code comprehension.

In this controlled experiment, we selected six code snippets from real open-source C/C++ systems of different domains containing atoms of three different types (*Assignment as Value*, *Conditional Operator*, and *Logic as Control Flow*), being two snippets for each type of atom. We selected these types of atoms because they are commonly found in industrial practice [10]. We then manually refactored the code to remove the atoms from the snippets and build our tasks. We also changed the function to remove several internal code dependencies so that the function can get more straightforward, but we keep its primary structure. We executed the experiment with 30 developers. The developers were asked to read, understand, and specify the six tasks' output, three of them with atoms and three with no atoms. We used the Latin Square design to assign the tasks randomly and minimize learning effects. We then measure the completion time of tasks, as well as the number of incorrect answers. Besides, through the eye tracker, we captured the developers' eyes coordinates while looking at the screen. These eye coordinates allowed us to build heatmaps, measure the number of times participants enter a particular area of the code, and measure the number of gaze transitions, which are the transitions of the eyes from one location of the code to another.

When analyzing all tasks together, we found that the presence of atoms of confusion statistically significantly increased the time required to understand the code by 43.02%. We could not find a statistically significant difference for accuracy. When analyzing the tasks individually, two of the three analyzed atoms show a statistically significant increase in time. Regarding accuracy, none of them showed statistically significant differences. The heatmaps show that the regions that receive most of the attention are precisely the ones where the atoms are placed. The regions with atoms have an increase of 36.8% gaze transitions. These results confirm and add a new perspective to the previous study [4].

In conclusion, our findings reinforce that atoms of confusion cause confusion by hindering developers' performance and code comprehension. They make the maintenance activity more time-consuming and require more visual effort. We also show that the eye-tracking methodology seems promising, revealing a new perspective not seen in previous works.

In summary, this paper provides the following contribution: *An empirical controlled experiment with the use of an eye tracker demonstrates that the presence of atoms of confusion leads to more time to understand code and more visual effort.*

2 EXAMPLE

Atoms of confusion frequently appear in open source repositories. A previous study on their prevalence on 50 projects found more than 109 thousand occurrences of 11 out of 12 atoms types considered, including the ones we focus on this paper: *Conditional Operator* with 98% of prevalence, *Assignment as Value* with 94% of prevalence, and *Logic as Control Flow* with 50% of prevalence [10].

Figure 1 illustrates two code snippets from function `printf` written in C programming language. The left-hand side snippet contains a function with an example of the atom of confusion called *Conditional Operator*. [1] Notice the two nested ternary operators. At the right-hand side, a developer refactored the function to remove the *Conditional Operator* atom by replacing it with a regular `if-else` statement.[2] For examples of *Assignment as Value* and *Logic as Control Flow* see Figure 4.

According to Gopstein et al. [5], there is empirical evidence that atoms of confusion impact developers' performance, causing misunderstanding, and that their removal makes code easier to understand. However, to have an even more realistic understanding of such atoms on code comprehension, we need to assess other dimensions besides time and accuracy to draw more insights. In this scenario, eye-tracking devices come at hand to allow us to locate where and for how long subjects are looking at the screen [17]. These devices have been used before in the code comprehension field (e.g., conditional compilation with `#ifdefs`) along with time and accuracy to provide additional information related to visual effort [7, 12].

Thus, to gather more information on how atoms impact code comprehension, we use an eye-tracking camera to analyze how those atoms influence the focus of attention. By triangulating the time, accuracy, and focus of the developers, we can better understand the impact of the atoms. For instance, it helps us to confirm whether the regions of the code that receive most of the attention are the places where the atoms are positioned, how the focus of attention changes interacting with code elements, and whether removing those patterns alleviates developers' effort from a visual perspective not considered in previous works.

[1] https://github.com/narnat/printf/commit/6b4f1fe6ef454f1a2c820e88582844151dcb8e6c
[2] https://github.com/narnat/printf/commit/385233be85dd400fc17be255b529e5b0cf14e29b

With Atom

```
1.  int print_int(va_list ap, params_t *params)
2.  {
3.      long long int i = params->l_modifier ? va_arg(ap, long long)
4.          : (params->h_modifier ? (short)va_arg(ap, long)
5.          : va_arg(ap, int));
6.
7.      ...
8.  }
```

No Atom

```
1.  int print_int(va_list ap, params_t *params)
2.  {
3.      long l;
4.
5.      if (params->l_modifier)
6.          l = va_arg(ap, long);
7.      else if (params->h_modifier)
8.          l = (short int)va_arg(ap, int);
9.      else
10.         l = (int)va_arg(ap, int);
11.     ...
12. }
```

Figure 1: A transformation removing an atom of confusion (Conditional Operator).

3 STUDY: CONTROLLED EXPERIMENT

We now present our controlled experiment. We present the settings, experimental units, execution, and data analysis procedures.

3.1 Study Settings

In this experiment, we analyze programs written in C with atoms of confusion and with no atoms of confusion using an eye-tracking camera to investigate the effects of the presence of the atoms concerning time, accuracy, and focus of attention in solving "specify correct output" tasks from the point of view of developers in the context of code comprehension.

We focus on the following research questions:

- *RQ_1: To what extent do atoms of confusion affect task completion time?* To answer this question, we measure the total time developers need to solve each task. Thus, our first null hypothesis ($H1_0$) is: there is no significant difference between the time required to understand code with no atoms of confusion (the control treatment) and code with atoms of confusion (the treatment under investigation);
- *RQ_2: To what extent do atoms of confusion affect task accuracy?* To answer this question, we measure the number of errors committed by developers while solving a task. Thus, our second null hypothesis ($H2_0$) is: there is no significant difference in the number of errors committed by developers when understanding code with no atoms of confusion (the control treatment) and code with atoms of confusion (the treatment under investigation).
- *RQ_3: To what extent do atoms of confusion affect the focus of attention?* We measure the number of gaze points captured using an eye-tracking system and generate heatmaps. The heatmaps of the aggregated tasks solved by the developers help us visualize possible differences in attention. Our third null hypothesis ($H3_0$) is: there is no significant difference in the number of gaze points when analyzing code with no atoms of confusion (the control treatment) and code with atoms of confusion (the treatment under investigation). Also, we measure the number of gaze points inside the area of the atom of confusion and compare it with the number of points in the region modified to remove the atom.

We use a Latin Square design of order two with replicas [2] in our experiment, mainly because our goal is to compare two treatments and block two variables: (a) participant skill and engagement; and (b) two sets of comprehension tasks. Each Latin Square replica comprises two participants and two sets of comprehension tasks (representing the columns of each square, i.e., ST_1 and ST_2). Each set of tasks has three tasks, one containing a *Logic as Control Flow*, one containing an *Assignment as Value*, and one containing a *Conditional Operator*. Figure 2 presents the design of our experiment.

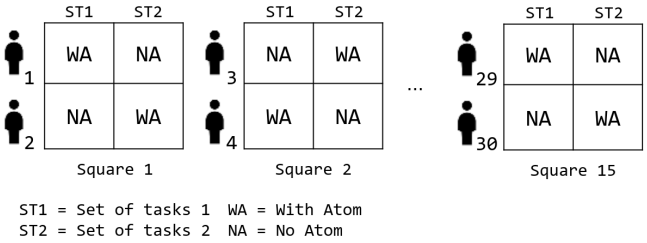

Figure 2: Design of the experiment using Latin squares.

We also randomly set the treatments that each participant should use in ST_1 and ST_2. Each treatment appears once in each row and column. This way, we block the two sources of variability: the participants and the two sets of tasks [9, 15]. The design also leads to one replica for each Latin Square (increasing the number of errors' degrees of freedom) [14].

3.2 Experimental Units

In total, 30 subjects participated in the experiment, 9 undergraduates, 17 masters, and 4 Ph. D. at Pontifical Catholic University of Rio de Janeiro (PUC-Rio). Out of the 30 subjects, 25 were men, while 5 were women. Although we have used code structures common to most programming languages, prior knowledge in C, C++, or Java was required to participate in this experiment. Initially, we recruited 33 subjects. However, we had to discard 3 of them. We discarded the 33rd subject because we could not run the experiment with the last Latin Square, lacking one subject. Then, we discarded

another subject because he answered the cell phone while participating in the experiment. This way, we also had to discard the other participant on his *Latin Square*.

For the experiment execution, we have prepared a laptop with a system developed to control the experiment's execution flow. The system is a web application written in *Python* using the *Django Web Framework*. All captured data was stored in a *PostgreSQL* database to posterior analysis. The system showed the functions to the participants automatically when they finish a task based on the previous Latin Squares randomization. Figure 3 illustrates our setup. It shows the two possible configurations. The first configuration should be presented to the participants who (a) solve the first set of tasks (ST_1) comprising code with atoms of confusion, and (b) solve the second set of tasks (ST_2) comprising code with no atoms of confusion. Both sets contain three tasks: that is, ST_1 contains the tasks $T1$, $T2$, and $T3$, while ST_2 contains the tasks $T4$, $T5$, and $T6$. In the second configuration, we invert our treatments, according to the Latin Square design (Figure 3).

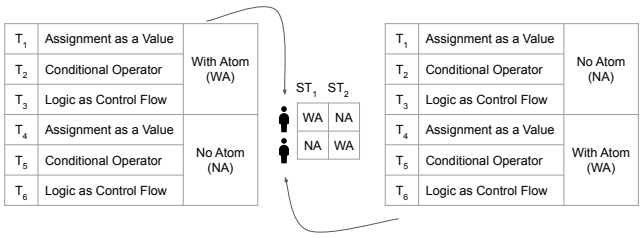

Figure 3: Structure of the experiment in terms of experimental units. We have two Sets of Tasks (ST_1 and ST_2) that comprehend Tasks 1 to 6 (T_1, T_2, .., T_6). Half of the tasks are with atoms (WA) and half with no atoms (NA).

To create the experiment's tasks, we mined the code of open source projects to find functions containing atoms of confusion. The selected functions were adapted and simplified to make them self-contained, fit them to our research scope, and reduce the time required for completion by participants. We have tasks with three different types of atoms (see Figure 4): *Assignment as Value*, *Conditional Operator*, and *Logic as Control Flow*. Each task has one or more `printf` statements; each participant's assignment is to specify the correct output to be printed on the standard output when the code snippet is executed. We have selected six functions in total, and for each selected snippet, we created a refactored version by removing the atom of confusion and use it as a control treatment group. The tasks have less than 30 LOC, so that scroll down could be avoided.

The applied transformations used for the tasks consisted of maintaining the same code except that the atom was replaced with a more clarified code fragment. There is not only one way to perform the transformation. Thus we followed the proposed transformations in [10] and proposed and evaluated in [4].

3.3 Execution and Data Analysis Procedures

We first randomly assigned the treatments (With Atom × with No Atom) to the 15 Latin square replicas. When a new participant

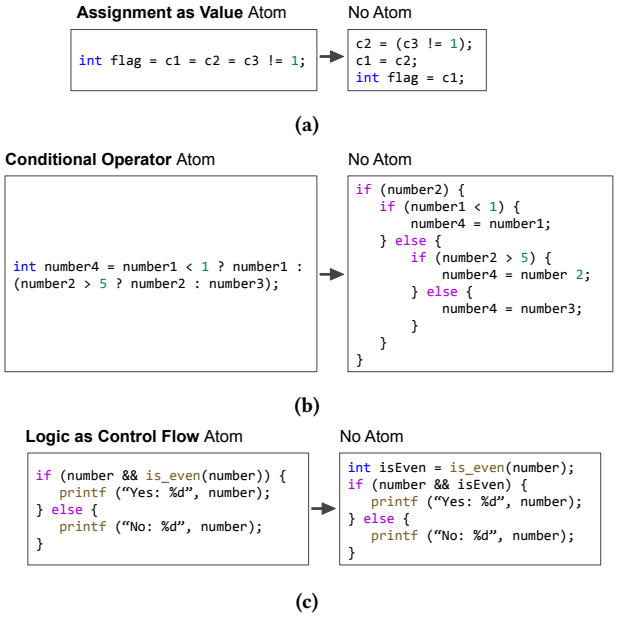

Figure 4: Code snippets with the three types of atoms evaluated in this study. At the left-hand side, we have the atom of confusion (treatment group) and at the right-hand side we have the same code with no atoms (control group).

starts the experiment execution, the system automatically assigns him to the next available row in the Latin Square's list. Before starting the tasks, for each participant, the eye-tracking device is re-calibrated. This procedure is required because the calibration varies according to the participant's physical characteristics and other external characteristics such as the use of glasses, positioning, and distance of the chair relative to the computer, participant's height, among others. We have plugged the device at the bottom of the laptop's screen to capture the points where the participant is looking. Then, we stored the captured eye gaze points in a database. Before executing the two sets of tasks, each participant completed a Warm-Up with just two elementary functions that did not contain atoms of confusion to understand how the flow of the experiment works and get acquainted with the eye tracker equipment setup.

Once the Warm-Up is successfully done, the system allows the participant to execute the main tasks. When a participant believes to have the correct answer, he or she should click anywhere on the screen, and the system will pause the experiment and show a screen that allows him or her to submit the answer.

If the participant's answer is wrong, the system increments the error count for this execution and allows the user to get back to the task or enter a new answer. Once the participant provides the correct answer, it allows him or her to go to the next task. During the tasks' execution, the participants are allowed to pause anytime by clicking anywhere on the screen. The system collects the start and end time for the execution of each task and the complete experiment. It also stores the number of wrong answers, the number of pauses and their duration and each provided answer. We considered the execution completed only when all tasks are correctly solved. To

avoid any inter-subject contamination, we instructed participants not to share answers or explain anything about the experiment to others.

We proceeded with an exploratory data assessment and tested the hypotheses introduced in Section 3.1. Since our dataset does not follow a normal distribution, we used the Kruskal-Wallis test.

We analyzed the eye gaze perspective using heatmaps, and we measured the number of gaze transitions, along with other metrics associated with how gaze points are located inside or move between regions of the code. Heatmaps are static graphical representations in colors, and the intensity of the color represents the concentration of gaze points over a particular area. We aggregated the heatmaps of the same task performed by 15 distinct participants. The number of gaze transitions adds a more dynamic perspective of analysis, expressing how many transitions of the focus of attention occurred in the code. The eye tracker camera captures about 80 sample points per second, and tasks took from a few seconds to a few minutes to be answered. To ease the analysis, we computed the median of the coordinates x and y of these 80 points in one second. This coordinate is the *focus of attention per second*. The transition of the eyes from one focus to another is the gaze transition. We analyzed the chronological sequence of the focus points in the code lines of all participants in the same task. To simplify the analysis, we considered only transitions that occur at least 1/3 times. At least 1/3 of the participants must have performed the same transition allowing us to observe a pattern.

4 RESULTS AND DISCUSSION

In this section, we present the results of our research questions along with a discussion.[3]

4.1 RQ_1: To what extent do atoms of confusion affect task completion time?

The boxplots of Figure 5 show some descriptive statistics related to the total time spent by the participants to conclude the different tasks.

The Aggregate boxplot represents an aggregation of the tasks, either with atoms or with no atoms. When aggregated, the total time median when analyzing code with atoms of confusion (294.5 seconds) is 65.4% greater than the median when analyzing code with no atoms of confusion (178 seconds). The observations related to the total time to conclude all tasks, when analyzing code with no atoms of confusion, lie between 74 and 778 seconds; in contrast, the observations when analyzing code with atoms of confusion lie between 91 and 736 seconds. There is only a small overlapping between the two "boxes", which leads to some evidence that understanding code with atoms of confusion is more *time consuming* than understanding code with no atoms of confusion.

We tested our first null hypothesis and found evidences for rejecting it ($p{-}value = 0.001972 < 0.05 = \alpha$). This leads to the first conclusion of this study that atoms of confusion increases the time required to solve the tasks.

[3] All results are available at the GitHub repository website: https://github.com/easy-software-ufal/Atoms-of-Confusion-Experiment-Data

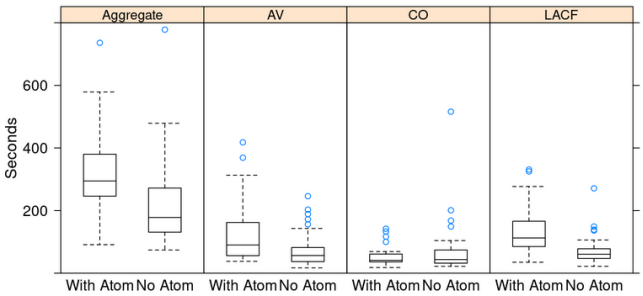

Figure 5: Time to conclude the tasks. Aggregate = aggregation of all tasks; AV = *Assignment as Value*; CO = *Conditional Operator*; LACF = *Logic as Control Flow.*

Under an individual perspective, considering the hypothesis test for each type of atom of confusion separately, we found diverging results. Two atoms of confusion allowed us to reject the null-hypothesis $H1_0$, particularly, the atoms *Assignment as Value* (p-$value = 0.008$) and *Logic as Control Flow* and ($p{-}value = 0.0001$). In both cases, they favored the code with no atoms.

Even though the size of the code can interfere with time, making it longer to look at more elements is not necessarily true. For instance, in Figure 4, the number of elements added to remove the atoms is considerably higher, especially in the *Conditional Operator*. These results suggest that the developers tend to achieve slightly higher productivity in terms of time in code with no atom rather than in code with atom. The slight increase in the time for the *Conditional Operator* with no atom can be explained by the relatively large amount of code added to remove such atom (see Figure 4b).

All the three evaluated atoms were present in the catalog of Gopstein et al. [4] and were evaluated by them. The only atom that showed distinct results from theirs was the *Conditional Operator*, which in their work showed statistical differences, but we could not observe such effect. The discrepancy might be due to other factors involved, such as methodology. For instance, we have used code from real projects, we assigned fewer tasks to participants, and the participants were not exposed to the same code with atom and with no atom. Besides, in our case, it is also possible that both codes with and with no atoms are confusing due to nesting. The fact is that in the version with no atom, for *Conditional Operator*, we have many more lines of code, which can affect time.

Previous studies have investigated the prevalence of atoms on real projects. Among the investigated atoms, *Conditional Operator*, *Logic as Control Flow*, and *Assignment as Value* were on the top seven of the most frequent atoms in projects according to Gopstain et al. [5]. On Medeiros et al. [10], *Conditional Operator* and *Assignment as Value* were found on the top three most commonly used. Given that *Conditional Operator* is commonly used on both evaluations, and we found conflicting results with Gopstain et al. [4] regarding its real effects, we need more studies on this topic. According to Medeiros et al. [10], participants accepted patches to remove the three evaluated atoms in this study. However, at least a patch of each atom was also rejected.

Finding 1. The presence of atoms of confusion increases 43.02% in time required to understand code correctly.

4.2 RQ_2: To what extent do atoms of confusion affect task accuracy?

We followed a similar approach to investigate the second hypothesis, which relates to answers correctness or accuracy when analyzing code with atoms of confusion. We first carry out an exploratory data analysis. Figure 6 shows boxplots that present some descriptive statistics related to the number of submitted answers to conclude all tasks under an aggregated and individual perspective. Under an aggregated perspective, the median number of answers when analyzing code with atoms of confusion is slightly higher than the median number of answers in code with no atoms. This result means getting the task solved on the first try. They only differ in terms of discrepant values. The same applies to the atoms individually. Consequently, we cannot infer a sound conclusion about our second hypothesis $H2_0$.

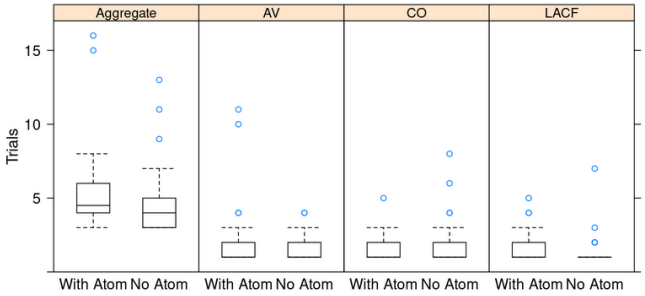

Figure 6: Number of trials to conclude all tasks. Aggregate = aggregation of all tasks; AV = *Assignment as Value*; CO = *Conditional Operator*; LACF = *Logic as Control Flow*.

One main reason why conclusive results could not be obtained by analyzing accuracy alone may rely on the fact that we have used simple tasks, with small pieces of code with a few operations. Thus, the number of submitted answers did not vary so much. In Gopstein et al. [4], for instance, even though the tasks were also small, the number of tasks answered by participants was higher.

Finding 2. We have not found evidence that the presence of atoms of confusion impacts the accuracy.

4.3 RQ_3: To what extent do atoms of confusion affect the focus of attention?

In the heatmaps, blue (cold) areas are areas of the code that did not receive much attention, while red (hot) areas represent the areas that received most of the attention. By the developers' distribution of visual attention, and how the focus of attention changes over distinct code elements, we can infer whether a particular code element is confusing. The analysis of heatmaps can assess the distribution of attention.The colors in the heatmap represent the relative concentration of gaze points over an area, and the higher the amount of points on it, the more intense the color gets. If the atom of confusion areas require more attention from the participants, they are likely to appear on more intense colors on the heatmaps, which is a phenomenon we are interested in investigating. Most importantly, we aim to verify the extent of the atoms' impact on the participants' attention. Heatmaps are exceptionally helpful and useful, giving the following reasons. First, they allow us to observe where the participants focused the most and with what intensity, and by doing so, we can see where most time and visual effort in task completion were spent. Second, we can also observe the areas of the code that do not catch the attention of participants, which is also a relevant information. Third, they allow us to perform a sanity check so that we can verify whether gaze samples were properly captured. To get more insights on this matter, we separated the areas of interest to perform more specific analysis. We have defined these Areas Of Interest (AOI) as the lines of code where the atom of confusion is present, and correspondingly, lines of the version with no atom. In Figure 4, we present the areas of interest in the code, which are the areas in which both codes differ.

Table 1: Summarizing Metrics. Bold font denotes a significant statistical difference with a significance level of 5%. Signal ↑ corresponds to an increase with atom while ↓ corresponds to a reduction.

Atoms	Time	Errors	Points in AOI	Entries in AOI	Gaze Trans.
AV	↑ 66.05%	↑ 46.7%	↓ 34.5%	↑ 17.9%	↑ 15.7%
CO	↓ 28.9%	↓ 9.3%	↑ 85.0%	↑ **121.2%**	↑ 93.7%
LACF	↑ 91.7%	↑ 16.3%	↑ 26.2%	↑ 36.7%	↑ 22.7%
Aggregated	↑ **43.0%**	↑ 16.2%	↑ 20.6%	↑ **49.3%**	↑ 36.8%

In Figures 7, 8, and 9, we see a comparison of the three heatmaps, one of each evaluated atom, showing how the attention is distributed over different parts of the program relative to time spent by the developers. Each heatmap is an aggregation of individual heatmaps of the participants who performed the same task.

In Figure 7, we have a more clear distinction in the heatmaps for *Logic as Control Flow* atom. We observe that the attention is mainly focused on one main region where the atom is located while when the atom is refactored, the developers directed their attention to two main regions, causing a higher distribution of attention over distinct parts. With atom, the total time was increased by 91.7%, and, in it, participants focused on the area of the atom with 26.2% more points. Participants also needed to enter the atom area 36.7% more and performed 22.7% more transitions. Those numbers give us and indication that code with atom is more confusing.

In Figure 8, we could not observe so much difference in the heatmaps for the *Assignment as a Value* atom. The developers tend to focus their attention on the atom region. When the atom is refactored, the developers focus on the relative lines of code that are extracted. However, we observe that the attention is more restricted and focused on a relatively smaller area at the right-hand side of the heatmap compared to a more sparse at the left-hand side. It is important to emphasize that the color intensity is relative to the amount of time to solve the code. However, according to Table 1, with atom, the time spent in the code increases along with errors. Gopstein et al. [4] have shown that this atom causes confusion, and by intense color on the atom area, we conclude that the participants

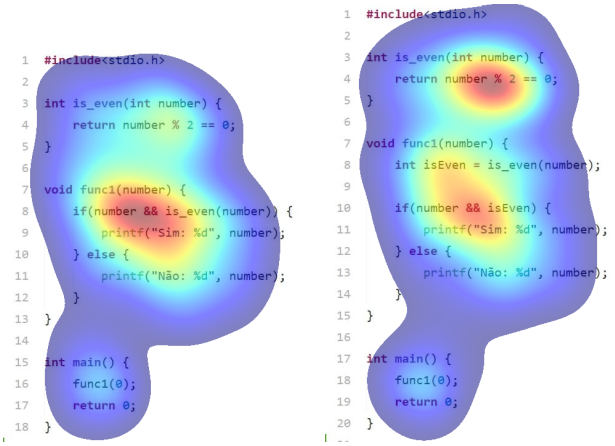

Figure 7: *Logic as Control Flow* - With atom vs. No atom.

are focusing on it because they are confused. However, in our results, the number of points in this area is decreased by 34.5%, which means that other parts of the same code were also confusing. This conclusion is supported by the fact that we have 17.9% more entries in the atom area and 15.7% more transitions in the code.

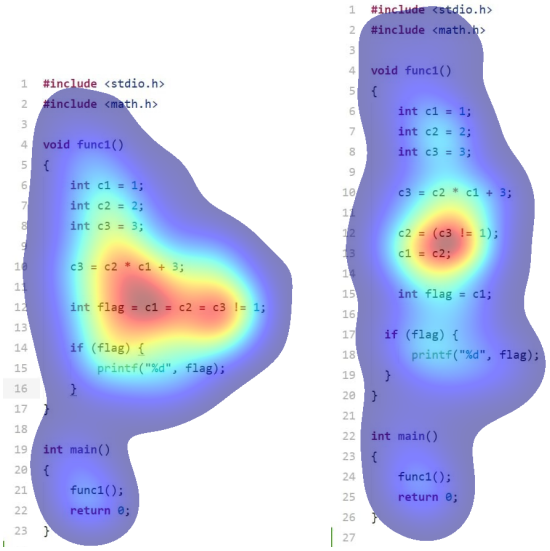

Figure 8: *Assignment as Value* - With atom vs. No atom.

In Figure 9, at the top, we observe that the attention is distributed more horizontally while at the bottom, vertically. The visual difference is not so clear, however, in terms of the concentration of points in the area of interest, there is a clear difference. Gopstein et al. [4] showed that the atom causes confusion, but, in our results, time and accuracy did not support those results. However, the eye gaze metrics indeed support those results. There is a higher concentration of points on the atom region, 85% more points, according to Table 1, which means that the atom region requires more attention. Also, there is a statistically significant increase by 121.2% in entries

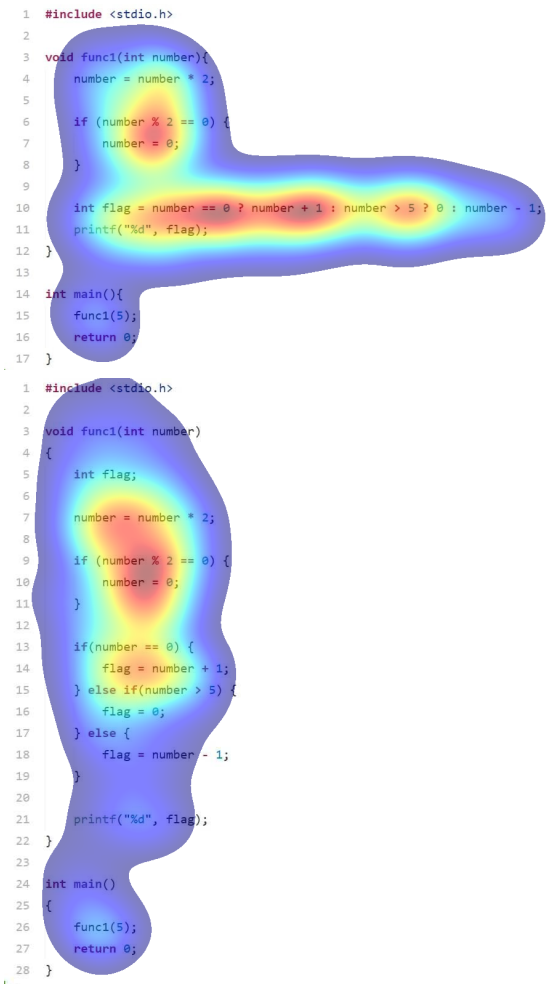

Figure 9: *Conditional Operator* - With atom (top) vs. No atom. (bottom)

in the atom area, which supports confusion associated with that area. It also does not seem to help understand other parts of the code since we have 93.7% more transitions with atoms.

The code with the *Conditional Operator* atom required, on average, less time to be solved but more visual effort due to some reasons. To discuss those reasons, we show an example of how two distinct participants solved the tasks. The first one solved the code with an atom, and the second one solved the code with no atom. For the participant who solved the task with an atom, we took the line of code of the atom and its correspondent graph of gaze transitions, which can be seen in Figure 10, at the top and left-hand side. Since the atom is located at only one line of code for this particular case, we examine the x-axis, representing how their eyes moved back and forth horizontally over the distinct elements in the function of time. The red color represents eyes looking inside the area of interest in the function of time. As we can see by the red colored line, this participant had to go back and forth several times at the line with the atom, going until the end and until the

middle. The participant reached almost the end of the line 3 times, represented by the highest peaks, 5 times about half of the line, and several smaller movements represented by smaller peaks. The participant spent 50 seconds going back and forth on the same line of the atom, which indicates confusion. On the other hand, the second participant analyzed a code with no atom (illustrated in Figure 10, at the top, right-hand side). Here, the difference is that the code with no atom has more lines of code. We can see a small variation on the same x-axis with considerable less effort in the area of interest (see the few red lines with respect to time).

We also analyze the y-axis representing the visual effort vertically in the code (see the bottom of Figure 10). With no atom, the participant makes more gazes on the y-axis, going up and down more often, with about 7 long peaks, and 5 of them touching the area of interest in about 30 seconds. However, the effort going horizontally at the line with atom may be higher since the participant stays in the area of interest for about 45 seconds. Thus, the comparison of the variations for x-axis and y-axis together shows a symptom of confusion introduced by the atom.

> **Finding 3.** The presence of atoms of confusion increases in 36.8% the number of gaze transitions, 49.3% the number of entries in AOI, and 20.6% the number of points in AOI.

5 THREATS TO VALIDITY

We now discuss potential threats to the validity of our controlled experiment. We distinguish between the threats using the classification of Wohlin et al. [21].

Conclusion Validity. Since our data do not follow a normal distribution, our hypotheses tests are based on a nonparametric test named Kruskal-Wallis. This test can determine if there are statistically significant differences between two or more groups of an independent variable on a continuous or ordinal dependent variable. Regarding the reliability of our measurements, the participant's eyes can gaze over areas out of the screen or out of the code, and the eye tracker can still capture those points. We observed, by analyzing the generated heatmap, that some participants tend to look into emptiness sometimes when they are thinking. However, this situation is likely to occur in both treatments, i.e., with atoms and with no atoms. Regarding the points captured, techniques are commonly employed to reduce their number, which the literature refers to as fixations. We did not reduce their number, but the raw data points can also be used to generate the heatmaps once they show the density of concentration of such points over distinct areas.

Internal Validity. There were some other threats to validity in this study that we realized during the experiment execution, such as the rotating chair used by the participants. This rotating chair can lead the eye tracker device to lose contact with the participants' eyes. Nevertheless, we minimized this threat because we calibrated the eye tracker device for each participant right before executing the experiment. The different potential luminosity between the two rooms environments we used to execute the experiment can also be considered a threat since they can change the sensibility of eye-tracking and expose different conditions to participants. To minimize this threat, we closed all curtains to avoid the external luminosity. Also, the size of the font and style might influence the

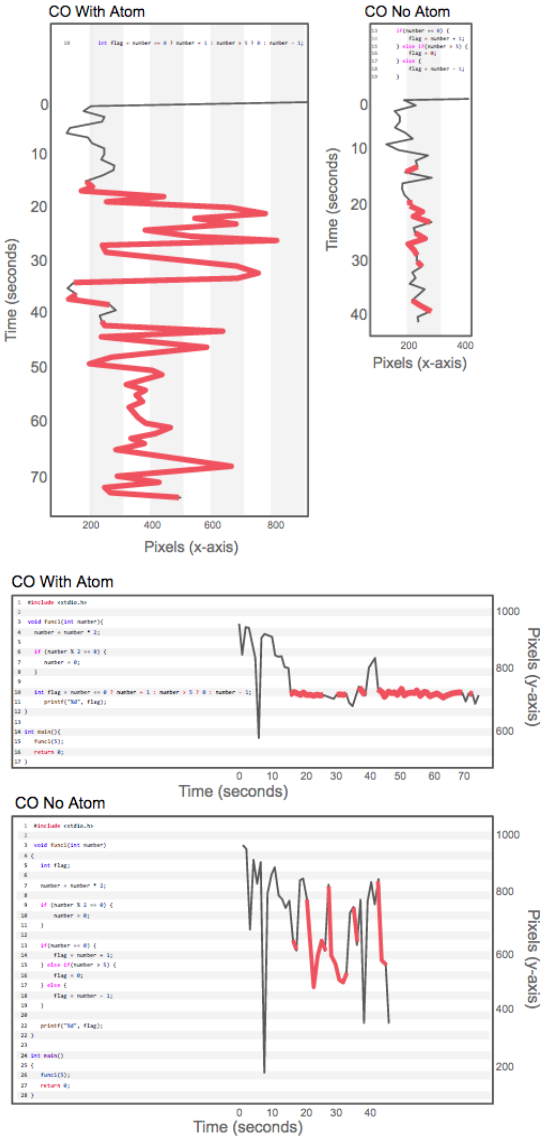

Figure 10: *Conditional Operator* transitions on x-axis (two graphs at the top) and y-axis (two graphs at the bottom).

participant's attention. To mitigate this threat, we have chosen a popular font style and a size that fits the screen. All tasks are displayed in the same font size, highlighted with a light theme, to mitigate possibles effects. The time duration of the whole experiment can also influence visual effort if it is too long. We mitigated it by using simple and only a few tasks to not let participants tired.

Construct validity. Our controlled study can involve threats regarding mono-operation bias or possible effects of interaction between treatments. To deal with and control these threats, we employed in our experiment the Latin Square design.

External Validity. In our experiment, we have used programs written in the C language, which might restrict the generalization capacity to other languages. However, we used simple code

snippets with constructs incorporated in several programming languages, such as variable declarations and assignments and `if-else` statements. Additionally, we cannot generalize the results of our experiment to other code comprehension scenarios—although we explored three different types of atoms of confusion: *Assignment as Value*, *Conditional Operator*, and *Logic as Control Flow*. Moreover, we cannot generalize our results to the professional developers' population since some of our experiment participants were graduate students. However, some studies argue in favor of using students to conduct controlled experiments [19, 22]. Our strategy in this paper was to conduct an empirical experiment that initially consisted of a seek on open source repositories to find functions to the experiment. Then we created a complete environment to make it possible to execute the experiment with the least possible interaction, and then the experiment allowed us to understand better some causal relationships of code comprehension of code containing atoms of confusion. This work aims to validate preexisting works favoring internal validity.

6 IMPLICATIONS

Our results reveal and reinforce what a previous study has claimed regarding the impact of atoms on time [4]. From an additional perspective, which includes eye-tracking, the presence of atoms can confuse, affecting developers' productivity, making them spend more time and misjudge code behavior, and bringing effects to their attention, which makes them focus on the atom regions.

The catalogs of atoms of confusion created by previous works are not yet widespread among software developers. Thus, developers need to be aware of any possible atoms of confusion, which appear as bottlenecks in code comprehension. This way, we invite and suggest the community to give a greater diffusion of these atoms and the negative effects of their presence on the code.

These results also motivate the development of plugins and tools in IDEs and code editors that can make it easier for programmers to detect and refactor atoms of confusion. A good practice is also to make project managers and senior developers more concerned about these atoms to be more restrictive when doing code reviews and accepting contributions (e.g., merging pull requests in GitHub) from other developers.

The methodology we adopted by using an eye tracker to assess visual attention is a promising alternative to analyzing code comprehension in the presence of atoms of confusion. It reveals how a particular atom can change the amount or direction of the attention, its distribution, and on this perspective, we have a more realistic idea of the impact of atoms. Previous works do not explore the eye perspective in the context of atoms of confusion.

Last but not least, in this work, we have found empirical evidence that atoms of confusion increase time to comprehend the code and that some types of atoms are more prejudicial than others. Thus, we encourage researchers, the academic community, and practitioners to conduct more controlled experiments, especially considering the types of atoms of confusion not explored in this paper.

7 RELATED WORK

Code comprehension is often explored in computer science, particularly in the software engineering field, since a significant portion of the maintenance effort is dedicated to understanding existing software. A considerable amount of effort to properly understand code can be a problem not only for the business environment but also for education since code comprehension tasks are widely used in assessments, mainly in the first years of graduation [16, 20].

Some prior works [4, 5, 10] have studied some particular small code patterns which have a big potential to cause misunderstandings. These code patterns were named *atoms of confusion* [4, 5], or *misunderstanding patterns* [10]. Such patterns can even lead to the introduction of bugs [5, 10]. Medeiros et al. [10] have exposed a case of bug introduction related to the atom of confusion *Dangling Else* in a function used on a codec library called OpenH264.

Gopstein et al. [4] have performed an experiment with 73 participants and showed empirically that these atoms of confusion could lead to a significantly increased rate of misunderstanding when compared with code without atoms of confusion. As they affirmed [4], people and machines often draw different conclusions about the behavior of a piece of code. These different conclusions can naturally lead to bugs. Still, according to Gopstein et al. [4], atoms of confusion can also cause diminished productivity, faulty products, and higher costs. They have also cataloged 15 atoms of confusion and provided a methodology for empirically deriving these complex atoms. In their study [4], they have conducted two experiments. One of them is similar to ours. However, we use an eye-tracking device to assess the impact of atoms of confusion on developers' attention. We observed an increase of 36.8% of gaze transitions in code snippets with atoms. In their experiment, Gopstein et al. [4] instructed subjects to step through the program as if they were the computer, execute each instruction in their mind, and record the standard output of the program. They also modified the experimental programs to include a *printf* after every control flow operation and otherwise frequently enough to gather information from the subject. Our experiment followed a very similar approach. This approach was mixed with an eye tracker device to record the subjects' eye movements.

Other works [1, 3, 6, 8, 11] have studied the correlation and effectiveness of visual attention tracking on code and text understanding. They have shown that a longer fixation on any specific part of the code can indicate a higher cognitive workload trying to understanding the text meaning. However, to the best of our knowledge, no studies with eye tracking were performed to investigate the impact of atoms of confusion in code comprehension.

Gospstein et al. [5] showed a strong correlation between atoms of confusion and bug fix commits and a tendency for atoms of confusion to be commented. They also observed a higher rate of security vulnerabilities in projects with more atoms. They did it by selecting 14 of the most representative and popular open-source C and C++ repositories to measure the prevalence and significance of atoms of confusion. They correlate the occurrence of these atoms on the selected projects with some external factors like bugs and comment rates. They showed that projects with more atoms tend to have more CVEs (Common Vulnerabilities and Exposures) and bugs by domain. In this experiment, they did not analyze developers' performances or asked them about the code; they searched for atoms of confusion in existing code. They analyzed the correlation between the rate of atoms on these repositories with bugs' presence and whether these codes were refactored, and the atoms were removed

from the code, among other questions. In our study, we include the measurement of how long it takes to developers understand code extracted from real open-source projects and which part of the code they fix their eyes to have more accurate data about the difference in code understanding with and with no atoms.

Another study conducted by Medeiros et al. [10] tried to better understand the occurrence and relevance of atoms of confusion by mining some repositories seeking for the occurrence of these atoms and also applied a survey with 93 developers. They found that the atoms studied (92%) are highly used in practice and that developers agreed that 6 of the studied atoms hinder code understanding, representing 50% of the 12 atoms studied. They also showed that the atoms are not frequently cited on guidelines. Our experiment tests three types of atoms and allows us to compare if they cause code misunderstanding in practice. Moreover, it allows us to catalog which atoms cause confusion and propose a tool to indicate and refactor them. Besides, we followed a more objective approach by using an eye-tracker, which allowed us to compare the perception of programmers with quantitative data about real confusion generated by the atoms.

8 CONCLUDING REMARKS

In this paper, we presented a controlled experiment with an eye tracker with 30 participants to investigate the effects of atoms of confusion on code comprehension. In particular, we analyzed the atoms' effects on time, accuracy, and participants' attention while specifying the output of six code tasks adapted from real open-source systems. We found evidence that code with atoms of confusion is more time-consuming, unlike code with no atoms of confusion. We also learned that developers tend to focus their attention on particular areas of the atoms more often. We observed an increase by 36.8% of gaze transitions in code snippets with atoms, thus more visual effort.

In previous work, Gopstein et al. [4] showed that the presence of these patterns in the code affects developers' performance, i.e., time and accuracy, and increase code misunderstandings. We showed that other aspects related to code comprehension besides time and accuracy, namely attention, should also be taken into consideration being affected by the presence of atoms. This was clear by the increase in the number of points in AOI, entries in AOI, and the number of gaze transitions, indicating more effort when analyzing code with atoms of confusion. These nuances would not be possible without an analysis from the eye perspective. Therefore, we presented a perspective of analysis that contributes to more insights into the atoms' impact on the attention.

For the developers' community, we recommend avoiding writing code with atoms of confusion since they can hamper code comprehension. For the research community, we encourage more empirical studies investigating the impact of other particular types of atoms of confusion, since we have seen that some atoms can cause more negative effects than others. Other comprehension code scenarios and activities should also be explored, and other eye-tracking metrics should also be investigated. We believe that more empirical studies on this subject can contribute to extensive catalogs of atoms of confusion among software developers. We also recommend them to propose and implement refactorings in IDEs to remove atoms of confusion automatically.

REFERENCES

[1] Roman Bednarik and Markku Tukiainen. 2006. An Eye-tracking Methodology for Characterizing Program Comprehension Processes. In *Proceedings of the Symposium on Eye Tracking Research &Amp; Applications (ETRA'06)*. 125–132.

[2] George E. P. Box, J. Stuart Hunter, and William G. Hunter. 2005. *Statistics for Experimenters: Design, innovation, and discovery*. Wiley-Interscience.

[3] Teresa Busjahn, Roman Bednarik, Andrew Begel, Martha Crosby, James H. Paterson, Carsten Schulte, Bonita Sharif, and Sascha Tamm. 2015. Eye Movements in Code Reading: Relaxing the Linear Order. In *Proceedings of the International Conference on Program Comprehension (ICPC'15)*. 255–265.

[4] Dan Gopstein, Jake Iannacone, Yu Yan, Lois Anne Delong, Yanyan Zhuang, Martin K.-C. Yeh, and Justin Cappos. 2017. Understanding Misunderstandings in Source Code. In *Proceedings of the Foundations of Software Engineering (FSE'17)*. 129–139.

[5] Dan Gopstein, Hongwei Henry Zhou, Phyllis Frankl, and Justin Cappos. 2018. Prevalence of Confusing Code in Software Projects: Atoms of Confusion in the Wild. In *Proceedings of the Mining Software Repositories (MSR'18)*. 281–291.

[6] Peter Gordon, Randall Hendrick, Marcus Johnson, and Yoonhyoung Lee. 2006. Similarity-based interference during language comprehension: Evidence from eye tracking during reading. *Journal of Experimental Psychology: Learning, Memory, and Cognition*, 1304–1321.

[7] Ahmad Jbara and Dror G Feitelson. 2017. How programmers read regular code: a controlled experiment using eye tracking. *Empirical Software Engineering (EMSE'17)* 22, 3 (2017), 1440–1477.

[8] Marcel Just and Patricia Carpenter. 1965. A theory of reading: From eye fixations to comprehension. Psychological Review. *Psychological Review*, 329–354.

[9] Romero Malaquias, Márcio Ribeiro, Rodrigo Bonifácio, Eduardo Monteiro, Flávio Medeiros, Alessandro Garcia, and Rohit Gheyi. 2017. The Discipline of Preprocessor-based Annotations Does #ifdef TAG N'T #endif Matter. In *Proceedings of the International Conference on Program Comprehension (ICPC'17)*. 297–307.

[10] Flávio Medeiros, Gabriel Lima, Guilherme Amaral, Sven Apel, Christian Kästner, Márcio Ribeiro, and Rohit Gheyi. 2019. An investigation of misunderstanding code patterns in C open-source software projects. *Empirical Software Engineering (EMSE'19)* 24, 4 (2019), 1693–1726.

[11] Jean Melo, Claus Brabrand, and Andrzej Wkasowski. 2016. How Does the Degree of Variability Affect Bug Finding?. In *Proceedings of the International Conference on Software Engineering (ICSE'16)*. 679–690.

[12] Jean Melo, Fabricio Narcizo, Dan Hansen, Claus Brabrand, and Andrzej Wasowski. 2017. Variability through the Eyes of the Programmer. In *Proceedings of the International Conference on Program Comprehension (ICPC'17)*. 34–44.

[13] Roberto Minelli, Andrea Mocci, and Michele Lanza. 2015. I know what you did last summer-an investigation of how developers spend their time. In *Proceedings of the International Conference on Program Comprehension (ICPC'15)*. 25–35.

[14] Douglas C. Montgomery. 2006. *Design and Analysis of Experiments*. John Wiley & Sons.

[15] Márcio Ribeiro, Paulo Borba, and Christian Kästner. 2014. Feature Maintenance with Emergent Interfaces. In *Proceedings of the International Conference on Software Engineering (ICSE'14)*. 989–1000.

[16] Carsten Schulte, Tony Clear, Ahmad Taherkhani, Teresa Busjahn, and James H. Paterson. 2010. An Introduction to Program Comprehension for Computer Science Educators. In *Proceedings of the ITiCSE Working Group Reports*. 65–86.

[17] Zohreh Sharafi, Timothy Shaffer, Bonita Sharif, and Yann-Gaël Guéhéneuc. 2015. Eye-tracking metrics in software engineering. In *Proceedings of the Asia-Pacific Software Engineering Conference (APSEC'15)*. 96–103.

[18] Janet Siegmund. 2016. Program comprehension: Past, present, and future. In *Proceedings of the Software Analysis, Evolution, and Reengineering (SANER'16)*, Vol. 5. 13–20.

[19] Dag Sjoberg, Bente Anda, Erik Arisholm, Tore Dyba, Magne Jorgensen, Amela Karahasanovic, Espen Frimann Koren, and Marek Vokác. 2002. Conducting realistic experiments in software engineering. In *Proceedings of the International Symposium on Empirical Software Engineering (ESEM'02)*. 17–26.

[20] Leigh Ann Sudol-DeLyser, Mark Stehlik, and Sharon Carver. 2012. Code Comprehension Problems As Learning Events. In *Proceedings of the Innovation and Technology in Computer Science Education (ITiCSE'12)*. 81–86.

[21] Claes Wohlin, Per Runeson, Martin Höst, Magnus C Ohlsson, Björn Regnell, and Anders Wesslén. 2012. *Experimentation in software engineering*. Springer Science & Business Media.

[22] Martin Yeh, Yu Yan, Dan Gopstein, and Yanyan Zhuang. 2017. Detecting and Comparing Brain Activity in Short Program Comprehension Using EEG. In *Frontiers in Education Conference (FIE'17)*. 1–5.